

UC LINA 600 LINEAR ALGEBRA FOR GRAPHICS · MODULE 04 · LECTURE NOTES

Change of Basis

The most useful idea in the course.

MODULE OUTCOMES

- Explain what a change of basis does.
- Build a change of basis matrix from the basis vectors.
- Invert it, and explain the direction convention.
- Explain the view matrix as a change of basis.
- Avoid the standard direction confusion.

1 The setup

THE ARROW DOES NOT CHANGE; ONLY THE NUMBERS DESCRIBING IT DO

A point in a room has one position, and two people using different coordinate systems will assign it different numbers — and neither of them is wrong (Module 02 §1's two readings).

So a change of basis is a translation between two descriptions — a dictionary, not a movement — and nothing in the world moves when you apply one, which is worth saying because the matrices look exactly like the ones that do move things.

Which is the distinction that causes all the trouble in this subject: rotating an object and rotating the frame you describe it in produce inverse matrices, and the two look identical on screen (§4).

And the view matrix is firmly the second kind — the camera does not move the world; it re-describes the world in its own coordinates — which is why §3 exists and why `lookAt` contains an inverse.

2 Building the matrix

```

SUPPOSE a basis B has vectors b1, b2, b3, each
written IN WORLD COORDINATES.

BUILD  M = [ b1 | b2 | b3 ]   (as columns)

THEN M takes B-coordinates TO world coordinates:
      v_world = M * v_B

WHY: v_B = (x,y,z) means "x of b1 plus y of b2 plus
z of b3" -- and M*v_B combines the columns with
exactly those weights (Module 03 section 1). So the
construction is forced; there is nothing to
remember.

AND THE OTHER DIRECTION is the inverse:
      v_B = M^-1 * v_world

THE DIRECTION CONVENTION, worth fixing once and
never rethinking:
  name the matrix M_world_from_B.
  Then M_world_from_B * v_B = v_world reads as
  cancellation, and
  M_world_from_B * M_B_from_camera is obviously
  M_world_from_camera.

```

Name it **M_world_from_B** — then the subscripts cancel like units and you can never compose two of them backwards, because a mismatched pair is visible in the name before you run anything. **This naming convention eliminates the single most common error in the subject**, and it costs nothing but a habit. Gortler uses it; most graphics texts do not, which is why most graphics texts are confusing here.

AND FOR AN ORTHONORMAL BASIS THE INVERSE IS JUST THE TRANSPOSE, WHICH IS WHY FRAMES ARE BUILT ORTHONORMAL

If the basis vectors are unit length and mutually perpendicular, the matrix is orthogonal and $M^{-1} = M^T$ — which is free to compute and numerically exact, with no division and no conditioning problem.

So camera frames, tangent frames, and skeleton bone frames are all built orthonormal deliberately — not for elegance, but because inverting them is then a transpose, and inverting them is something you do constantly.

And it is exactly why Gram-Schmidt matters (Module 05 §4): given any basis at all, it produces an orthonormal basis spanning the same space, so you can always arrange to be in the easy case.

Plus the numerical point: a general matrix inverse accumulates floating-point error and a transpose does not — which matters in a loop running sixty times a second, where drift compounds into visible skew.

3 The view matrix

- **Build the camera's frame in world coordinates: a forward vector, a right vector, and an up vector, orthonormal** — which is Module 05's cross products and normalisation, and is where the handedness convention first bites (Module 06 §3).
- **Those three vectors as columns give $M_{\text{world_from_camera}}$ (§2)** — the matrix taking camera-space coordinates to world-space ones.
- **But the pipeline wants the opposite: the view matrix is $M_{\text{camera_from_world}}$, which is the inverse** — and because the frame is orthonormal, that inverse is simply the transpose (§2's callout).
- **Plus the translation**, which does not fit into a 3×3 matrix at all and is Module 07's subject — and which inverts by negating the camera's position, expressed in the rotated frame.
- **Which is the entire derivation of `lookAt`**, and it takes four lines once the convention is fixed — and it explains why every implementation of that function contains a transpose and a negated translation, which is otherwise mysterious.

4 The standard confusion

ROTATING THE OBJECT AND ROTATING THE FRAME ARE INVERSE OPERATIONS THAT LOOK IDENTICAL

Turning an object thirty degrees to the left, and turning your head thirty degrees to the right, produce the same image — and the matrices that express them are inverses of one another.

So a sign error here does not crash anything; it produces a scene that moves the wrong way when you drag the mouse, which is diagnosable only by knowing which of the two operations you meant to express — and by then you are an hour in.

And the fix is the naming convention (§2): a matrix called $M_{\text{camera_from_world}}$ cannot be applied in the wrong direction without the name visibly disagreeing with the code around it, which turns a runtime mystery into a reading error.

Which is this module's practical deliverable — the idea itself is simple and the bookkeeping is where all the time goes, so fix the bookkeeping first and the idea stays simple.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
3Blue1Brown, chapter 13 (free)	§§1 and 2 — change of basis, and the whose-coordinates question made visual.

Resource	What it covers for this module
Gortler, chapters 3 and 4 (library copy)	The whole module — and the frame-naming discipline of §2 comes from here, which is why this book is recommended over the more popular ones.
MIT 18.06, the change of basis lecture (free video)	§2 — the algebra, carefully.
Scratchapixel, the camera and lookAt articles (free)	§3 — the view matrix derived rather than presented, with code.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write one arrow's coordinates** in two bases and convert between them.
- 2 **Build a change of basis matrix** from three given vectors.
- 3 **Verify $\mathbf{v_world} = \mathbf{M} \mathbf{v_B}$** on several points.
- 4 **Invert it** and check the round trip to tolerance.
- 5 **Build an orthonormal basis** and confirm the inverse is the transpose.
- 6 **Measure the error** of a general inverse versus a transpose over 10,000 compositions.
- 7 **Adopt the from-naming convention** throughout your code.
- 8 **Derive lookAt** from scratch, in four lines.
- 9 **Compare yours** to a library implementation and account for every difference.
- 10 **Deliberately invert the view matrix** and describe how the camera then behaves.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does a change of basis do, and what does it not do?
- 2 Why do the two kinds of rotation look identical?
- 3 How do you build the matrix, and why is the construction forced?
- 4 State the naming convention and what it prevents.
- 5 When is the inverse a transpose, and why does that matter?
- 6 Why are frames built orthonormal?
- 7 Derive the view matrix in four steps.

- 8 Why does lookAt contain a transpose?
- 9 What does a direction error here look like at runtime?
- 10 What is this module's practical deliverable?